

CS2610: Computer Organization and Architecture

Lab

Lab 11: Branch Predictor

13th April 2026

Objective

The objective of this assignment is to handle control hazard, as we handled data hazard in the previous assignment.

1 Branch Prediction in a Two-Stage Pipeline

To handle control hazards due to conditional branches in a two-stage pipeline (S1: Fetch/Decode/Register Read, S2: Execute/Writeback), we implement a simple dynamic branch predictor based on a **2-bit saturating counter**.

1.1 Design Overview

Since decode is completed early, the processor identifies branch instructions in S1. Unlike deeper pipelines, a Branch Target Buffer (BTB) is not required because:

- The branch target can be computed directly using the PC and B-type immediate.
- Target calculation: $Target = PC + Imm_B$

Thus, prediction focuses only on **direction** (taken / not taken).

1.2 Branch History Table (BHT)

We use a **Branch History Table (BHT)** with the following properties:

- Number of entries: 64
- Each entry: 2-bit saturating counter
- Indexed using lower bits of PC (e.g., $PC[7:2]$)

1.3 2-bit Saturating Counter

Each BHT entry is a 2-bit FSM:

State	Prediction	Meaning
00	Not Taken	Strongly Not Taken
01	Not Taken	Weakly Not Taken
10	Taken	Weakly Taken
11	Taken	Strongly Taken

Update Rule

- Increment counter on taken (saturate at 11)
- Decrement counter on not taken (saturate at 00)

1.4 Pipeline Operation

Stage S1 (Fetch + Decode)

- Use PC to index BHT
- Read 2-bit counter
- Predict:
 - Taken if MSB = 1
 - Not taken otherwise
- If predicted taken:
 - Compute target: $PC + Imm_B$
 - Next PC = target
- Else:
 - Next PC = PC + 4

Stage S2 (Execute + Resolve)

- Evaluate branch condition
- Determine actual outcome (taken / not taken)
- Compare with prediction

Misprediction Handling

- If prediction is wrong:
 - Flush incorrect instruction in pipeline - don't do writeback/memory operation in Stage2
 - Redirect PC to correct target

BHT Update

- Update corresponding 2-bit counter based on actual outcome

1.5 Summary

This design provides a low-cost and effective solution for handling control hazards:

- No BTB required due to simple target computation
- 2-bit predictor improves accuracy over 1-bit schemes
- Small hardware overhead (64-entry BHT)

2 Evaluation and Submission Instructions

- Verify the implementation with all testcases of previous lab, and see effectiveness of branch predictor
 - Extra marks for a enhanced Branch Predictor design
1. The assignment must be completed individually, on top of the submission of Lab9.
 2. This assignment is a take home one, viva will be held in next lab.
 3. The following artifacts must be submitted:
 - Screenshots of all the modifications in each module.
 - Sample test code with the screenshot of this test code running.
 - The updated logisim circuit file.
 4. Compress all files into a single ZIP archive and submit it on Moodle.